

```

1  import re
2
3  from fastapi import FastAPI
4  from pydantic import BaseModel, Field, field_validator
5
6  from chunking import flat_chunks
7  from ingestion import DocumentInput, client, embed_chunks, embed_fn
8  from retrieval import build_context, build_prompt, retrieve_top_k
9
10
11  app = FastAPI()
12
13  # In-memory chunk store for the coding demo.
14  chunks = []
15
16  CAR_ALIASES = {
17      "comprehensive analysis review": "CAR",
18      "prior CAR": "previous CAR",
19      "rating justification": "rating rationale",
20  }
21
22
23  class ChatRequest(BaseModel):
24      query: str = Field(min_length=1, max_length=2000)
25
26      @field_validator("query")
27      @classmethod
28      def normalize_query(cls, value: str) -> str:
29          cleaned = " ".join(value.split())
30
31          if not cleaned:
32              raise ValueError("query must not be empty")
33
34          for alias, canonical in CAR_ALIASES.items():
35              pattern = r"\b" + re.escape(alias) + r"\b"
36              cleaned = re.sub(
37                  pattern,
38                  canonical,
39                  cleaned,
40                  flags=re.IGNORECASE,
41              )
42
43          return cleaned
44
45
46  class ChatResponse(BaseModel):
47      answer: str = Field(min_length=1, max_length=2000)
48
49      @field_validator("answer")
50      @classmethod
51      def validate_answer(cls, value: str) -> str:
52          cleaned = " ".join(value.split())
53
54          if not cleaned:
55              raise ValueError("answer must not be empty")
56
57          return cleaned
58
59
60  def call_llm(prompt):
61      response = client.responses.create(
62          model="gpt-5.5",
63          input=prompt,
64      )
65

```

```

66         return response.output_text
67
68
69 @app.post("/ingest")
70 def ingest(request: DocumentInput):
71     new_chunks = flat_chunks(
72         request.document_id,
73         request.text,
74     )
75
76     embedded_chunks = embed_chunks(
77         new_chunks,
78         embed_fn,
79     )
80
81     chunks.extend(embedded_chunks)
82
83     return {"chunks_indexed": len(embedded_chunks)}
84
85
86 @app.post("/chat", response_model=ChatResponse)
87 def chat(request: ChatRequest):
88     if not chunks:
89         return ChatResponse(
90             answer="No documents have been ingested."
91         )
92
93     top_chunks = retrieve_top_k(
94         request.query,
95         chunks,
96         embed_fn,
97         5,
98     )
99
100     context = build_context(top_chunks)
101     prompt = build_prompt(request.query, context)
102     answer = call_llm(prompt)
103
104     return ChatResponse(answer=answer)

```